

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Тема: UI-тестирование

Студенты
гр.4382

Н.А. Смирнов
А.В. Калинина
В.С. Бурая
А.А. Писаренко

Руководитель

А.М. Шевелева

Санкт-Петербург
2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Н.А. Смирнов

Группа: 4382

Тема практики: UI-тестирование

Задание на практику:

Разработка программного комплекса для автоматизированного UI-тестирования функциональных блоков веб-приложения. В рамках работы создается подробный чек-лист, включающий 10 тестовых сценариев, в общей сложности содержащих не менее 50 действий, а также выполняется их реализация на языке Java с применением библиотек Selenide и JUnit 5. Архитектура системы проектируется на основе паттерна Page Object Model и компонентного подхода, что позволяет разделить логику тестирования и описание элементов интерфейса, повышая удобство поддержки и масштабирования кода. Дополнительно осуществляется настройка системы логирования Log4j2 для отслеживания процесса выполнения тестов, организация командной работы через репозиторий GitHub и подготовка итоговой документации, включающей описание структуры классов и UML-диаграммы.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 18.07.2026

Дата защиты отчета: 18.07.2026

Студенты

Н.А. Смирнов
А.В. Калинина
В.С. Буря
А.А. Писаренко

Руководитель

А.М. Шевелева

АННОТАЦИЯ

В данном отчете рассматриваются проектирование и разработка системы автоматизированного UI-тестирования для образовательного веб-ресурса Stepik. В ходе выполнения работы были изучены функциональные возможности платформы и составлен чек-лист, включающий десять основных пользовательских сценариев. На языке Java реализован тестовый фреймворк на основе архитектурного паттерна Page Object Model, который позволяет отделить тестовую логику от деталей реализации интерфейса. Для разработки использованы библиотеки Selenide и JUnit 5, выполнена интеграция системы логирования Log4j2, а также подготовлена техническая документация, содержащая UML-диаграммы. Исходный код и материалы проекта опубликованы в репозитории GitHub.

SUMMARY

This report focuses on the design and development of an automated UI testing system for the Stepik educational web resource. During the project, the platform's functionality was analyzed and a checklist consisting of ten key user scenarios was created. A testing framework was implemented in Java based on the Page Object Model architectural pattern, which enables the separation of test logic from the user interface implementation details. The development process utilized the Selenide and JUnit 5 libraries, while the Log4j2 logging system was integrated to monitor test execution. In addition, technical documentation, including UML diagrams, was prepared. The source code and project materials were published in a GitHub repository.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 ПОСТАНОВКА ЗАДАЧИ	8
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	24
2.1. Описание программных классов и их методов	24
2.2. UML-диаграмма классов	26
2.3. Реализация взаимодействия компонентов.....	27
3 ИНДИВИДУАЛЬНАЯ РАБОТА	29
4 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	32
5 ОТЗЫВ РУКОВОДИТЕЛЯ	33
ЗАКЛЮЧЕНИЕ	34
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	36

ВВЕДЕНИЕ

Предметной областью данной практики является автоматизированное функциональное UI-тестирование веб-приложения. Выполнение проверок вручную требует значительных временных затрат и не исключает вероятность возникновения ошибок, связанных с человеческим фактором. Автоматизация тестирования таких элементов системы, как поиск, фильтрация и навигация, позволяет обеспечить стабильность работы ключевых функций приложения и корректность отображения данных после внесения изменений в программный код.

Цель практики заключается в разработке и реализации масштабируемой архитектуры автоматизированного тестирования, основанной на принципах объектно-ориентированного программирования. Особое внимание уделяется созданию универсального тестового фреймворка, позволяющего сократить дублирование кода, повысить удобство сопровождения тестов и обеспечить надежность проверок за счет применения современных средств автоматизации и проверенных архитектурных решений.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Выполнить анализ функциональных возможностей исследуемой платформы и сформировать чек-лист, включающий десять тестовых сценариев, охватывающих основные пользовательские процессы.

2. Спроектировать архитектуру программного решения на языке Java с использованием паттерна Page Object Model [1] и компонентно-ориентированного подхода, обеспечивающих разделение тестовой логики и элементов пользовательского интерфейса.

3. Реализовать автоматизированные тесты с применением библиотек Selenide [2] и JUnit 5 [3], настроить систему логирования Log4j2 [4] для отслеживания процесса выполнения тестов, а также подготовить техническую документацию, содержащую визуализацию структуры системы с помощью UML-диаграмм классов.

Кроме того, необходимо равномерно распределить задачи между всеми участниками.

Краткое описание вклада каждого участника:

Смирнов Н.А.: Базовая настройка проекта: создание Maven проекта, подключение зависимостей (Selenide, JUnit5, Log4j2), работа над общей архитектурой проекта, реализация UI-компонентов, добавление документации к компонентам страницы, настройка BaseTest, работа над чек-листом, проверка архитектуры кода и проверка соблюдения принципов Page Object Model, приведение классов компонентов страницы к единому виду (правильный порядок элементов класса), вынесение Selenide элементов в переменные, удаление неиспользуемых методов и импортов.

Калинина А.В.: Создание чек-листа, работа над общей архитектурой проекта, реализация Page Objects (HomePage, SearchResultsPage и т.д.), , добавление документации к объектам страниц, настройка логирования, подготовка README, приведение классов страниц к единому виду (правильный порядок элементов класса)вынесение Selenide элементов в переменные, удаление неиспользуемых методов и импортов.

Бурая В.С.: Тесты №1–5: навигация по страницам, поиск с фильтрами, фильтрация по ценовому диапазону, поиск для продвинутых с сертификатом, сброс текстового поиска "крестиком". Проверка стабильности выполнения тестов. Исправление и добавление методов. Удаление всех комментариев с действиями из тестов. Вынесение тестов с фильтрацией в отдельный класс.

Писаренко А.А.: Тесты №6–10: смена запроса с сохранением фильтра, несуществующий запрос с фильтрами, поиск по автору с переходом в профиль, фильтрация акционных курсов, добавление курса в избранное из поиска. Добавление assert-проверок и обработка исключений. Проверка работу всех тестов в связке. Исправление и добавление некоторых методов. Создание отдельного класса для работы с окнами. Вынесение всех магических чисел в константы.

1 ПОСТАНОВКА ЗАДАЧИ

В итоговом проекте имеются 10 тестов:

1. Текстовый поиск и навигация по страницам
2. Поиск с фильтрами
3. Фильтрация по ценовому диапазону
4. Поиск для продвинутых с сертификатом
5. Сброс текстового поиска «крестиком»
6. Смена запроса с сохранением фильтра
7. Несуществующий запрос с фильтрами
8. Поиск по автору с переходом в профиль
9. Фильтрация акционных курсов
10. Добавление курса в избранное из поиска

Тест №1: Текстовый поиск и навигация по страницам

Данный тест проверяет базовую работоспособность поиска по ключевому слову, корректный переход между страницами результатов через пагинацию и возможность открытия детальной карты курса.

Результаты действий представлены на рисунках (см. рисунки 1 – 4).

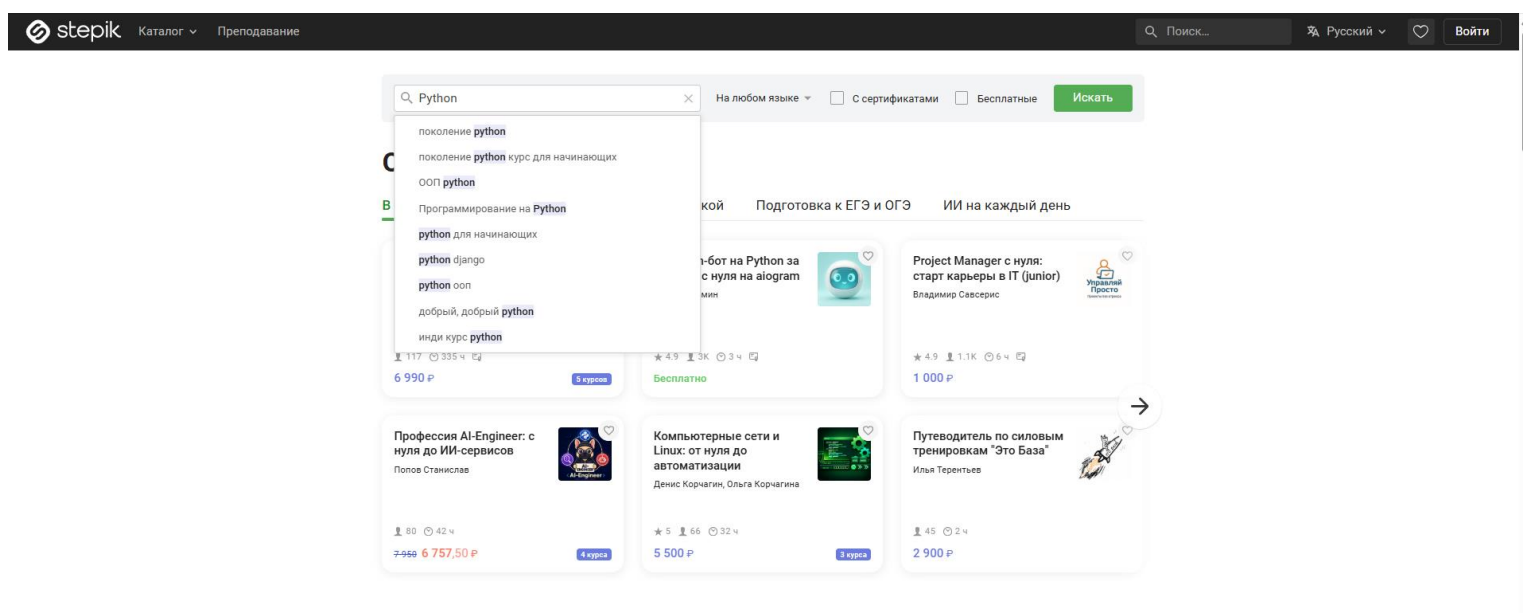


Рисунок 1 – Введение в поиск запрос «Python»

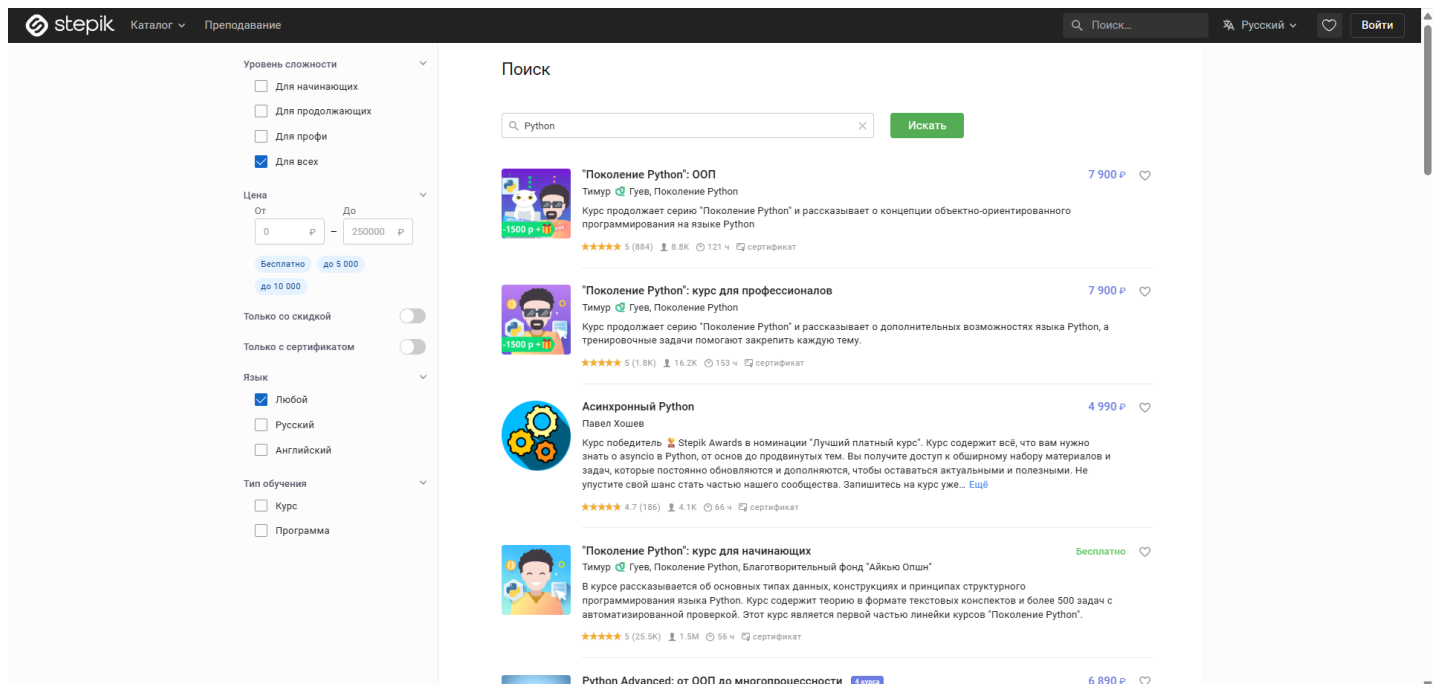


Рисунок 2 – Результаты поиска по запросу «Python»

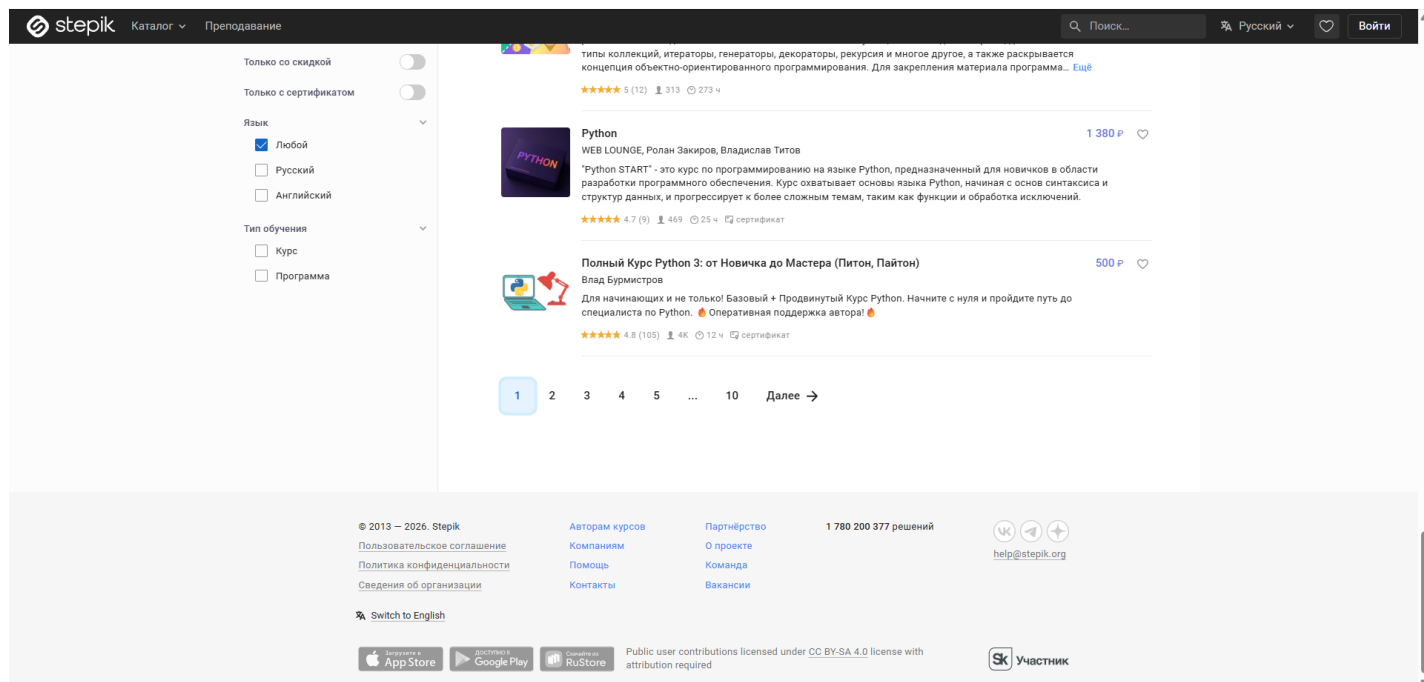


Рисунок 3 – Прокрутка вниз к пагинации

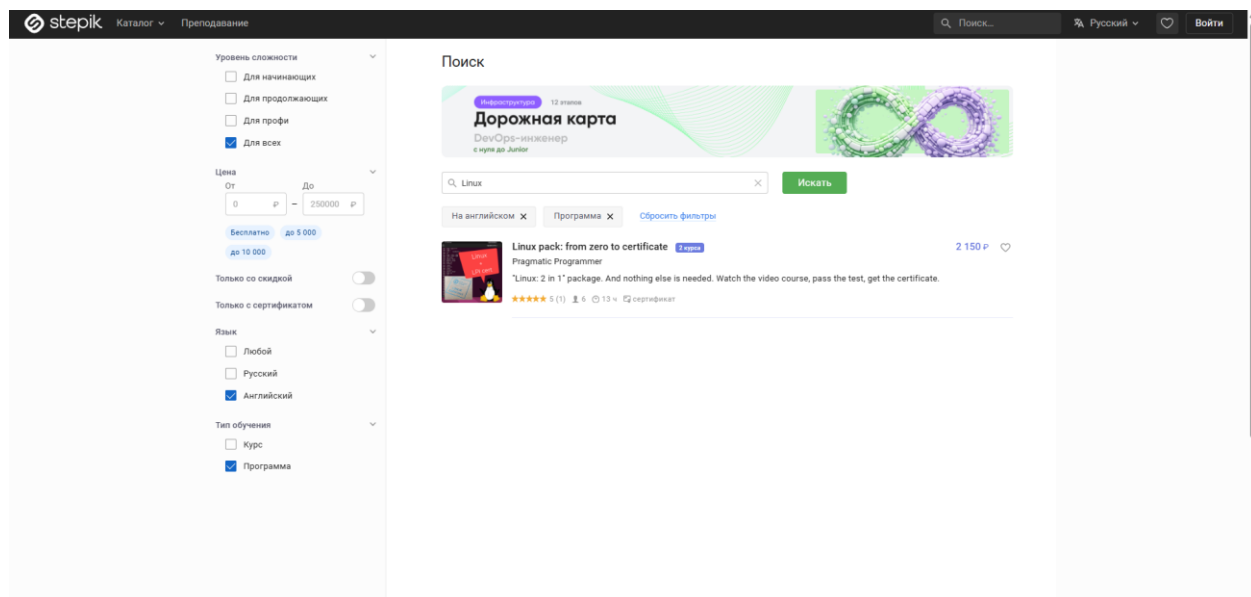


Рисунок 6 – Применение чекбоксов «Английский» и «Программа»

Тест №3: Фильтрация по ценовому диапазону

Данный тест подтверждает возможность подбора курсов в строго заданных границах стоимости, проверяя работу полей ввода минимальной и максимальной цены.

Результаты действий представлены на рисунках (см. рисунки 7, 8).

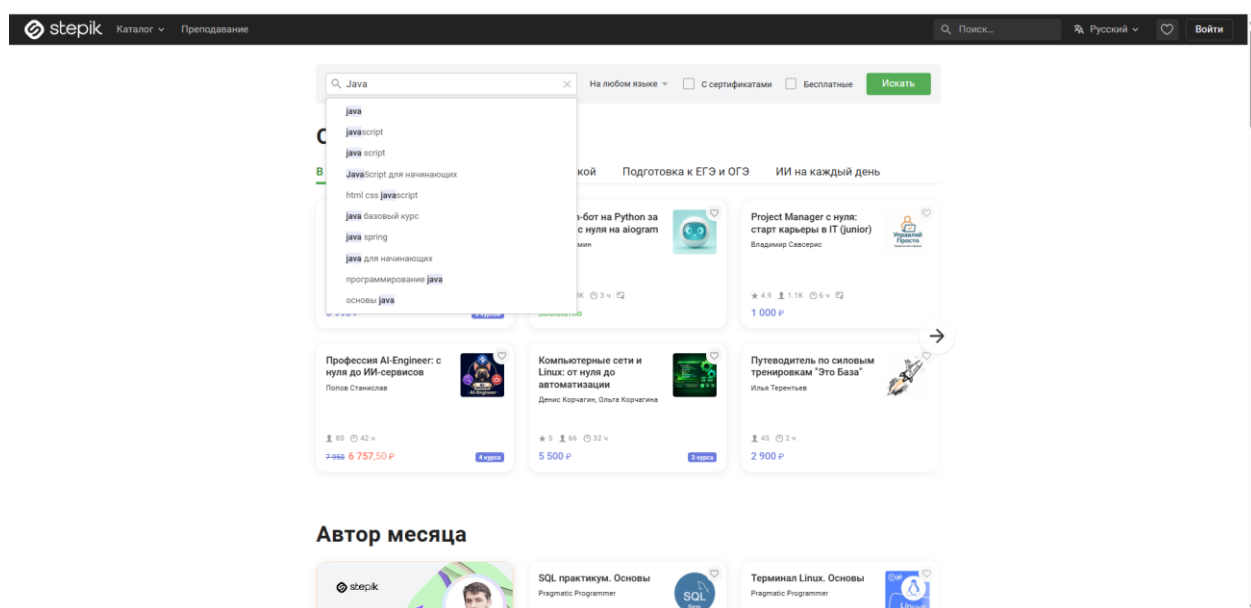


Рисунок 7 – Введение в поиск запрос «Java»

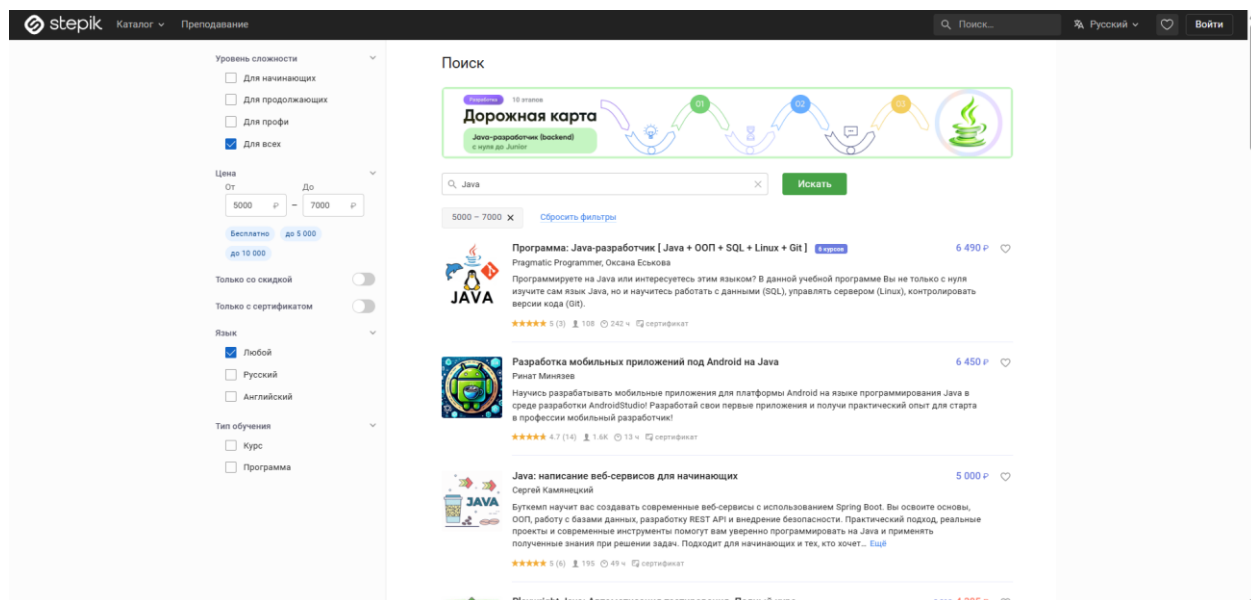


Рисунок 8 – Применение фильтра по цене

Тест №4: Поиск для продвинутых с сертификатом

Данный тест имитирует запрос профессионального пользователя, проверяя одновременную работу фильтра высокого уровня сложности и тумблера поиска курсов только с выдачей сертификата.

Результаты действий представлены на рисунках (см. рисунки 9 – 10).

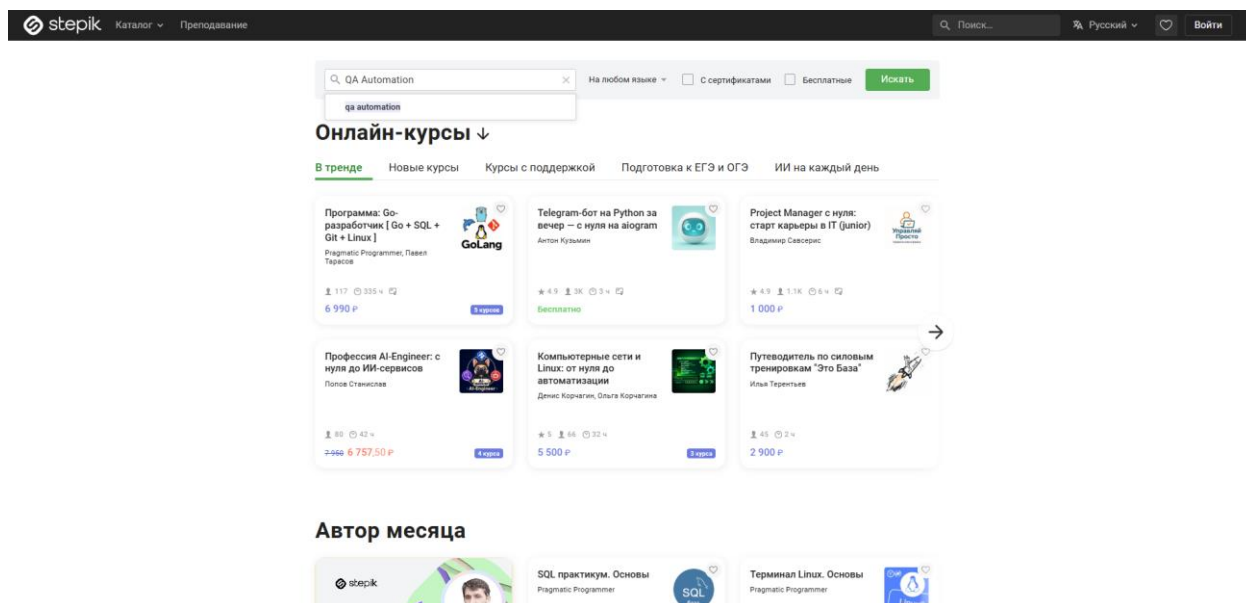


Рисунок 9 – Введение в поиск запрос «QA Automation»

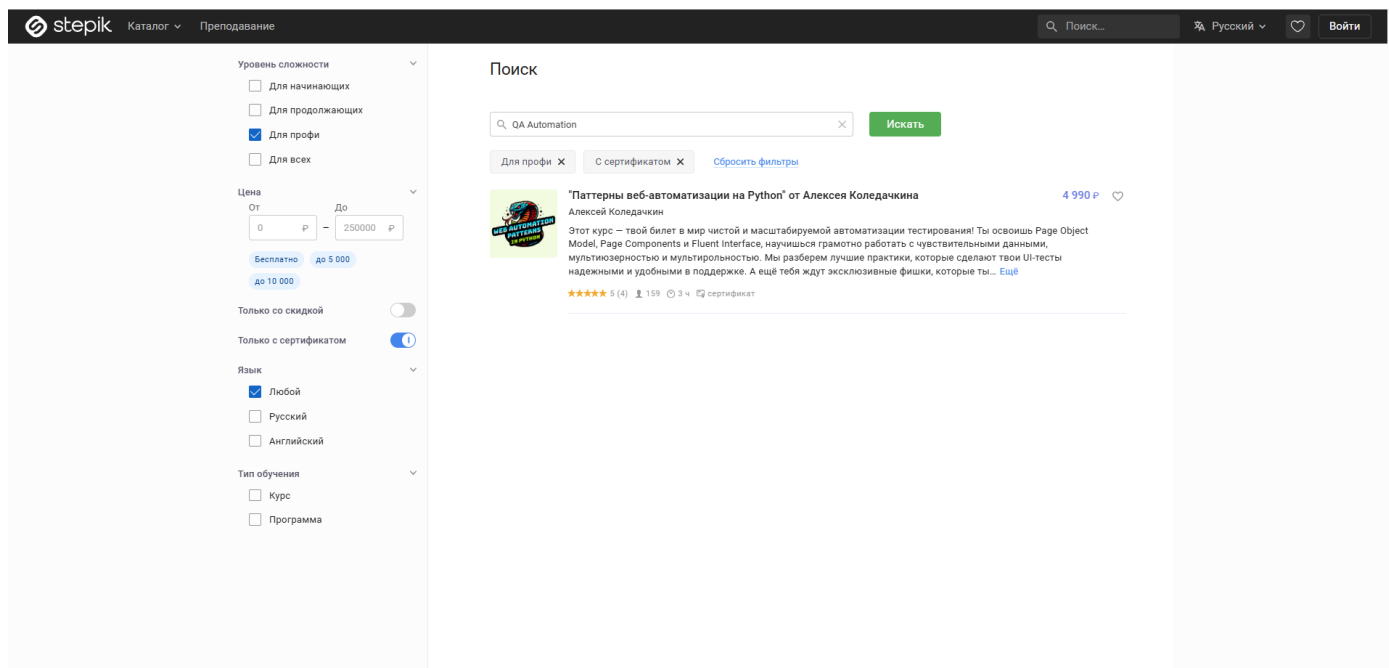


Рисунок 10 – Применение чекбоксов «Для профи» и «Только с сертификатом»

Тест №5: Сброс текстового поиска «крестиком»

Данный тест проверяет функциональность элемента очистки поисковой строки, контролируя удаление текста запроса и автоматический возврат каталога к общему виду.

Результаты действий представлены на рисунках (см. рисунки 11 - 13).

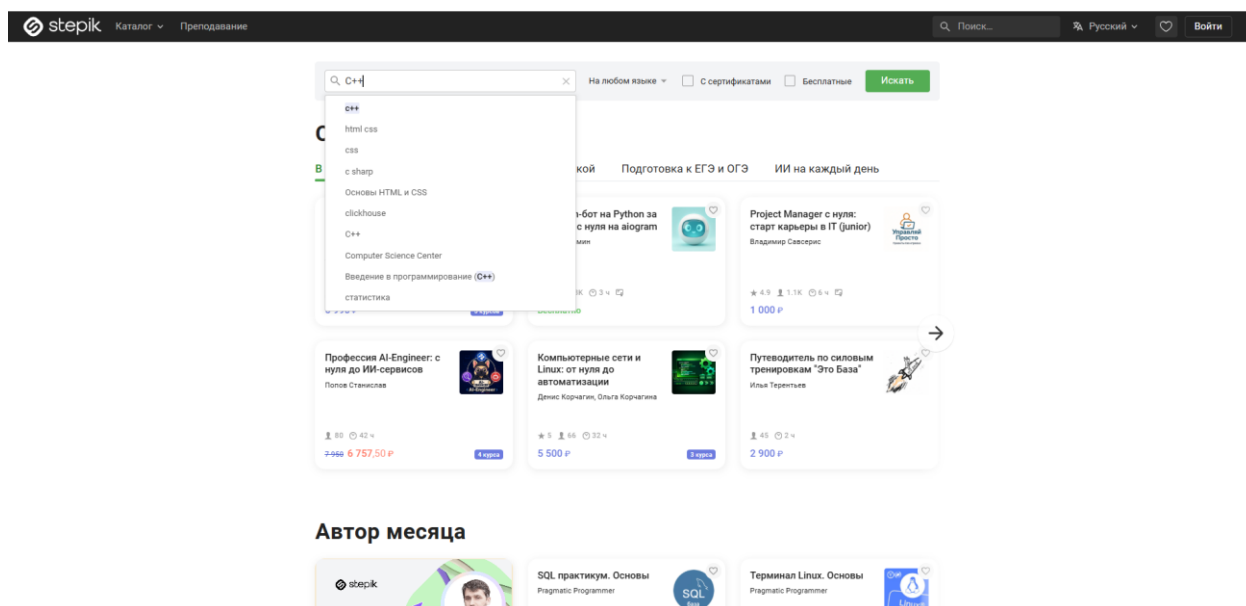


Рисунок 11 – Введение в поиск запрос «C++»

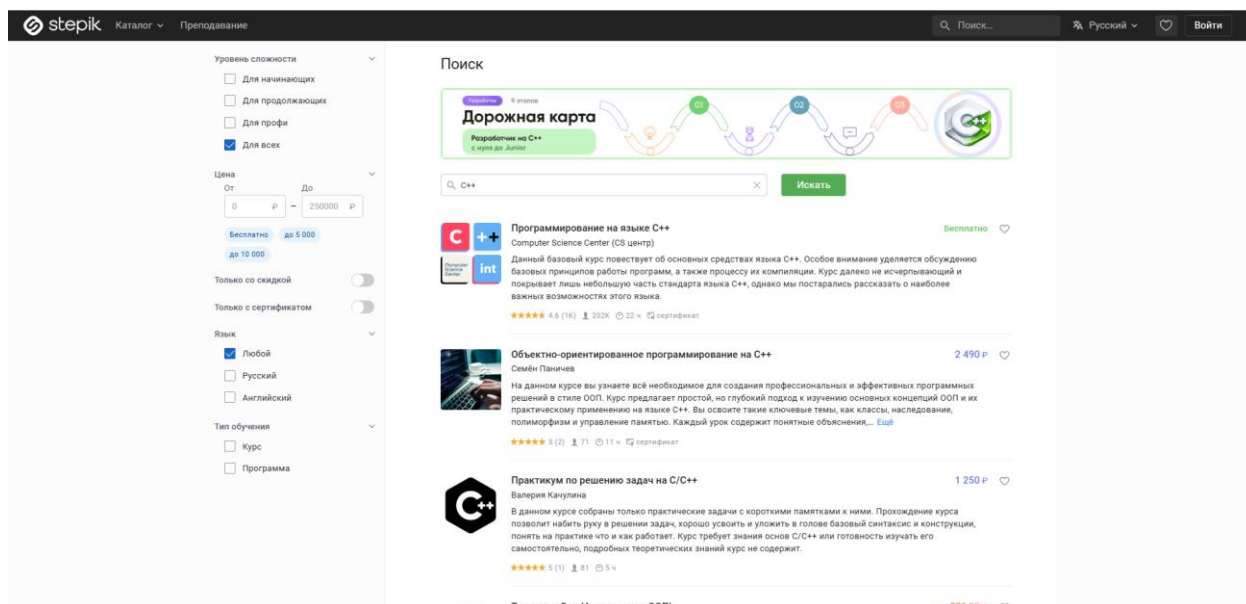


Рисунок 12 – Вывод всех доступных курсов

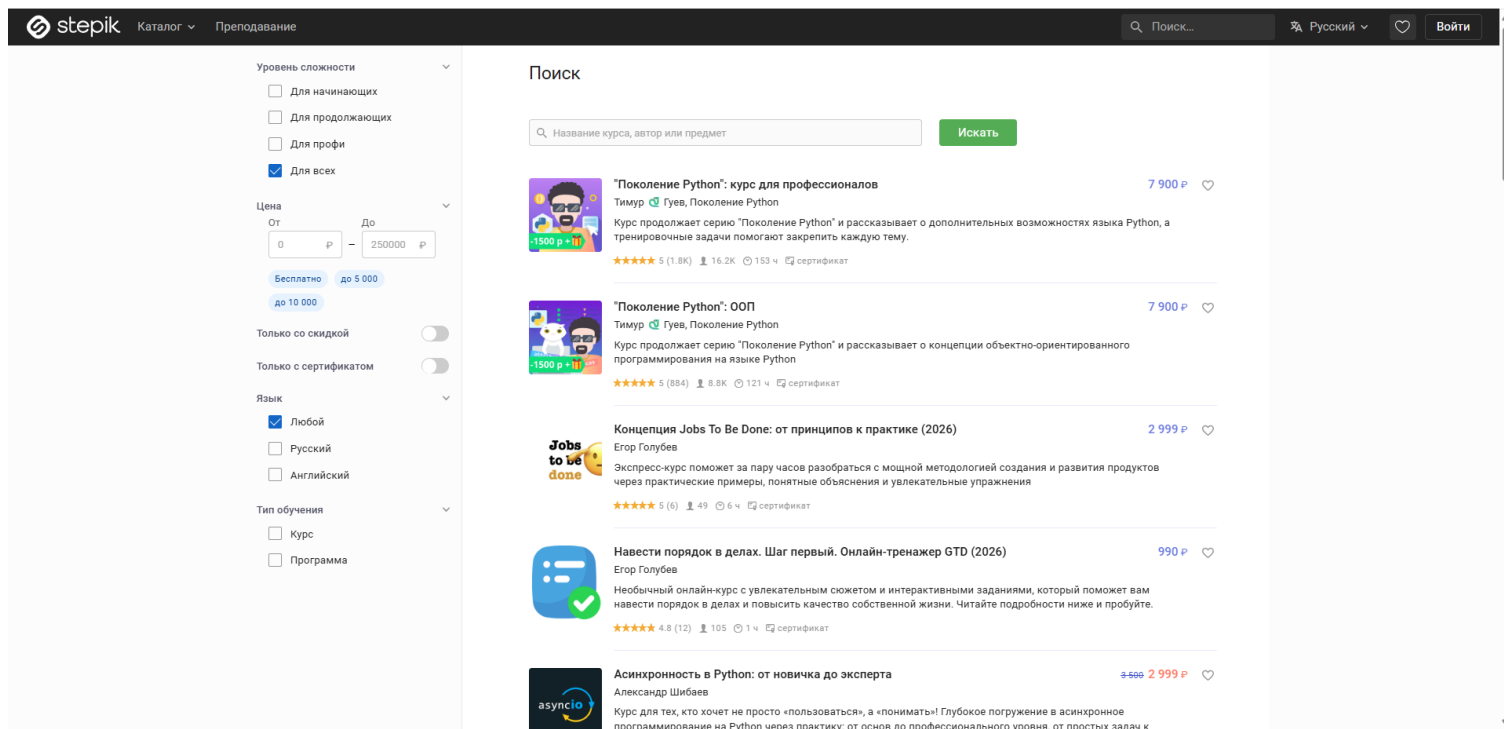


Рисунок 13 – Нажатие крестика

Тест №6: Смена запроса с сохранением фильтра

Данный тест работает с интеллектуальным поведением интерфейса, проверяя, остаются ли активными выбранные фильтры (например, язык) при вводе нового ключевого слова в поле поиска.

Результаты действий представлены на рисунках (см. рисунки 14 – 16).

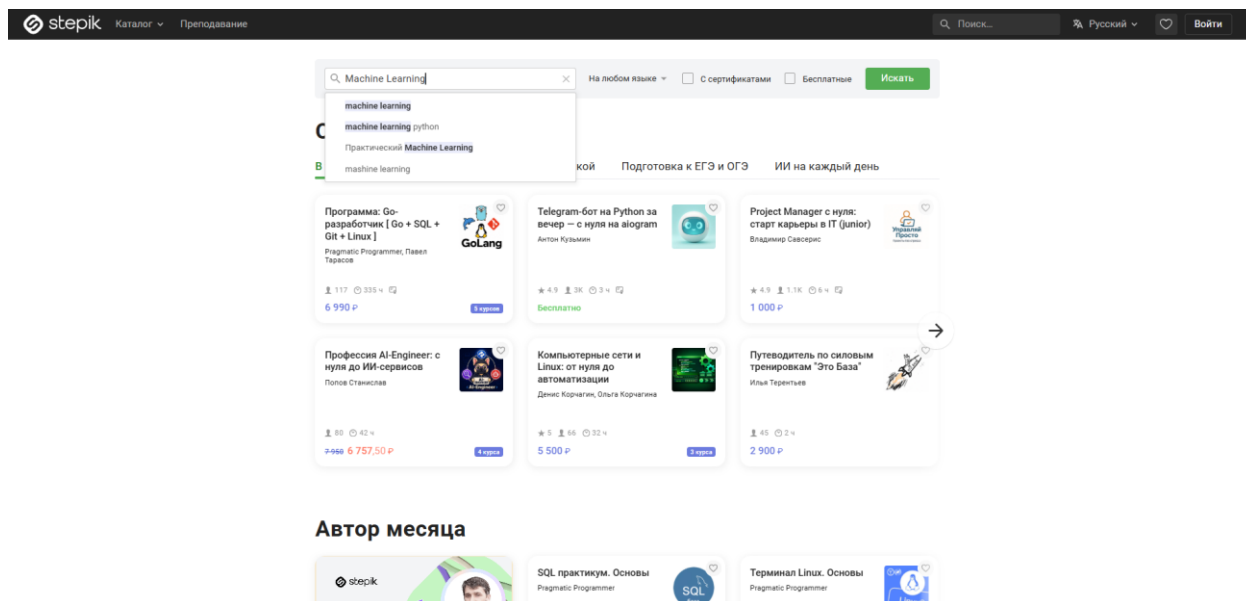


Рисунок 14 – Введение в поиск запрос «Machine Learning»

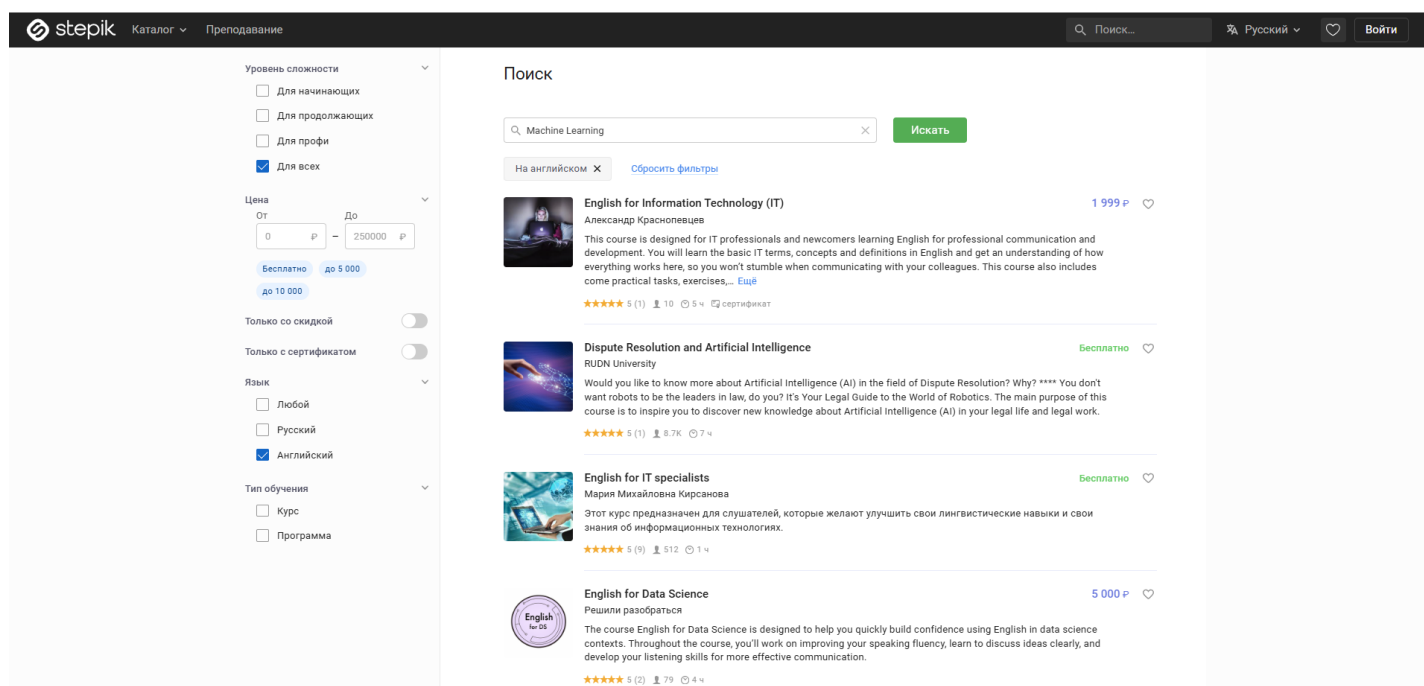


Рисунок 15 – Показ всех доступных курсов

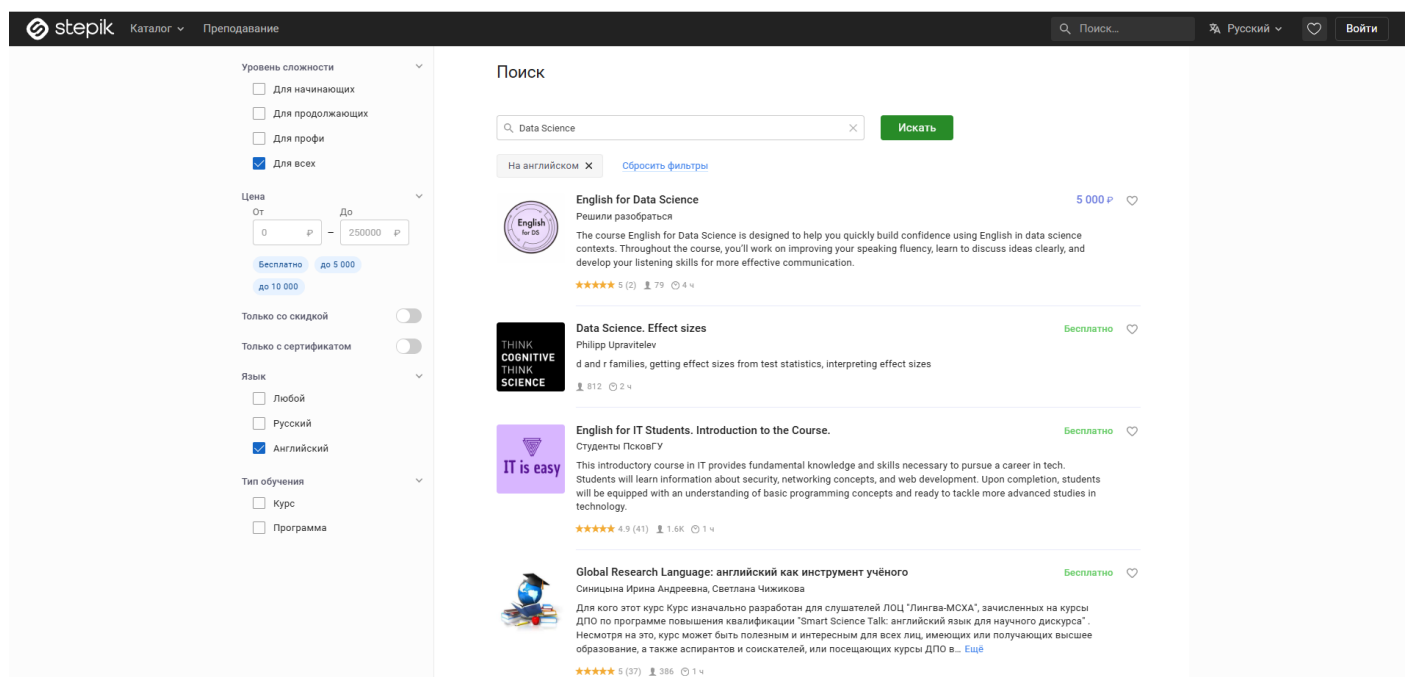


Рисунок 16 – Введение нового запроса «Data Science»

Тест №7: Несуществующий запрос с фильтрами

Данный тест проверяет корректность обработки негативного сценария, верифицируя появление сообщения «ничего не найдено» при вводе бессмысленного набора символов с примененными фильтрами.

Результаты действий представлены на рисунках (см. рисунки 17, 18).

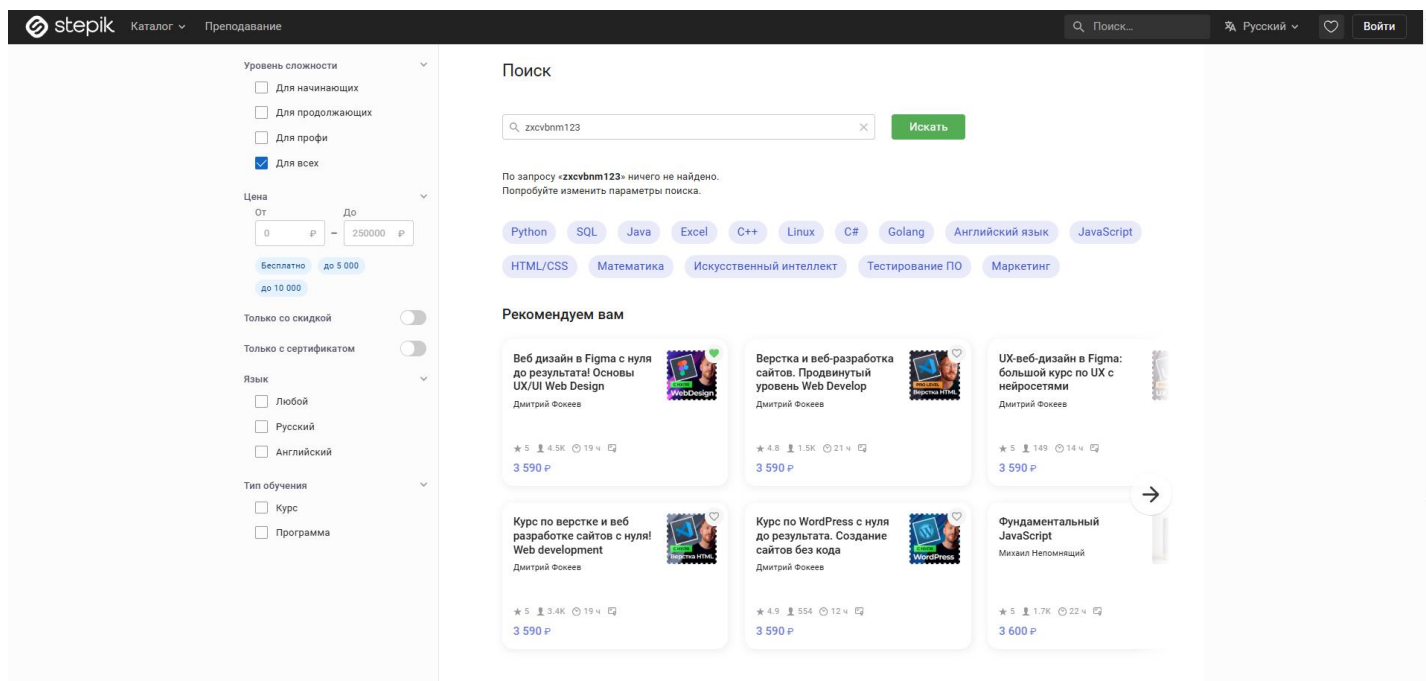


Рисунок 17 – Введение в поиск запрос «ячсми123»

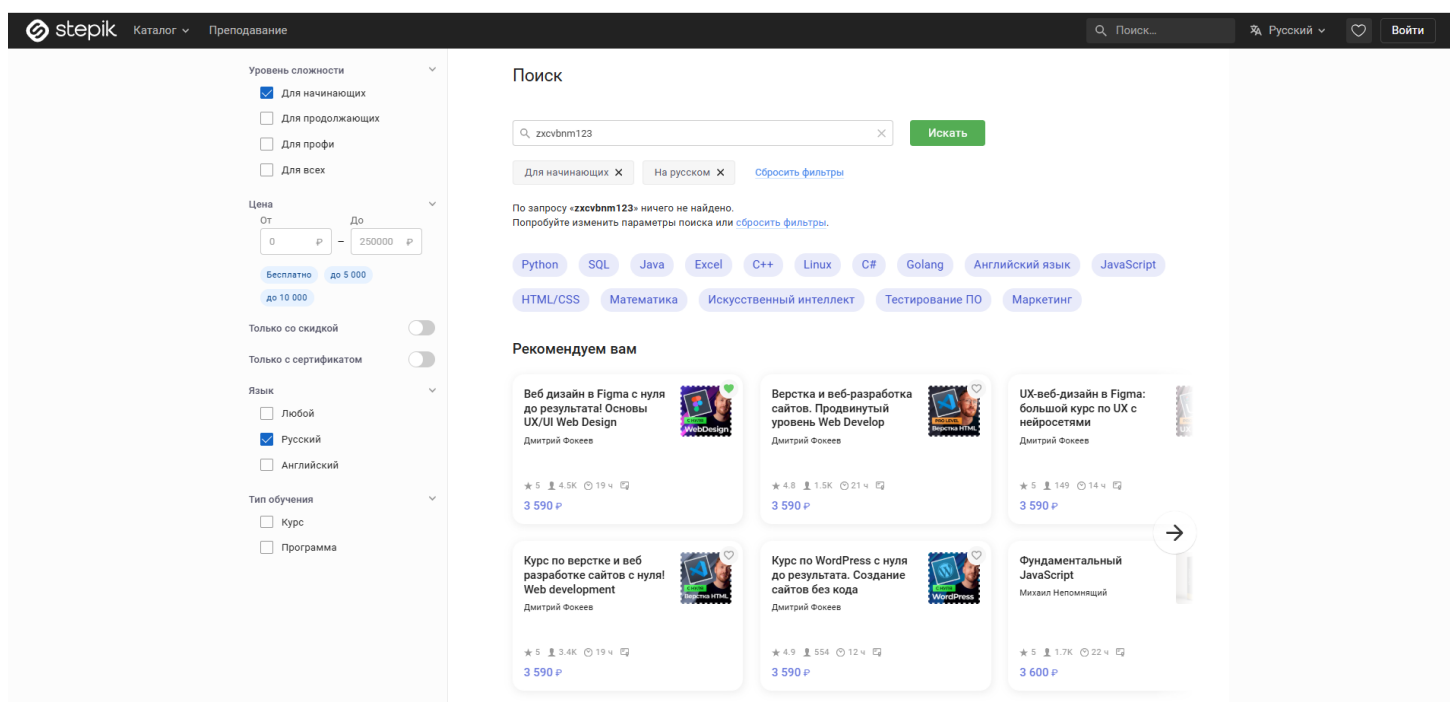


Рисунок 18 – Применение чекбоксов «Русский» и «Для начинающих»

Тест №8: Поиск по автору с переходом в профиль

Данный тест проверяет точность поиска курсов конкретного преподавателя и работоспособность ссылок, ведущих из карточки курса в личный профиль автора для просмотра отзывов.

Результаты действий представлены на рисунках (см. рисунки 19 – 22).

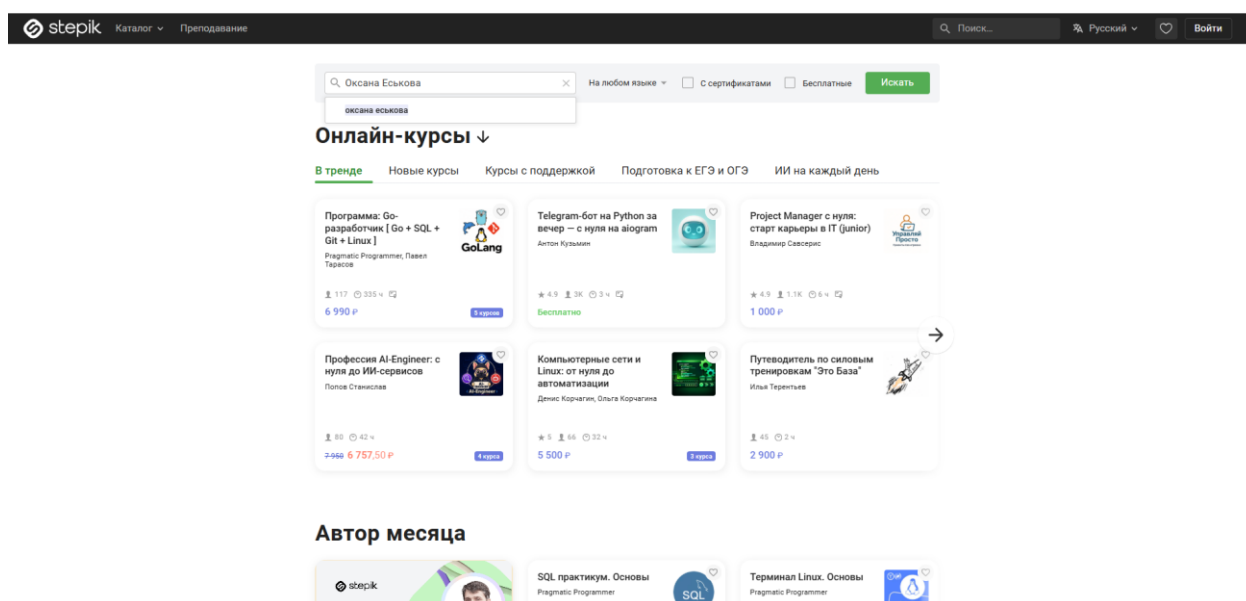


Рисунок 19 – Введение в поиск запрос «Оксана Еськова»

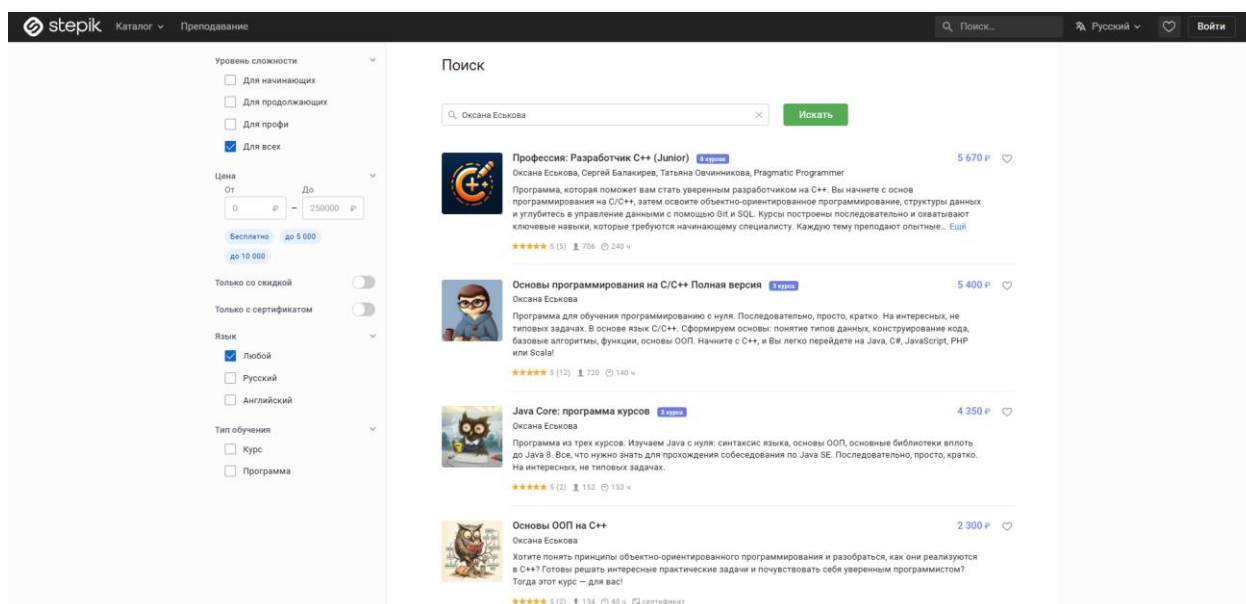


Рисунок 20 – Показ всех доступных курсов

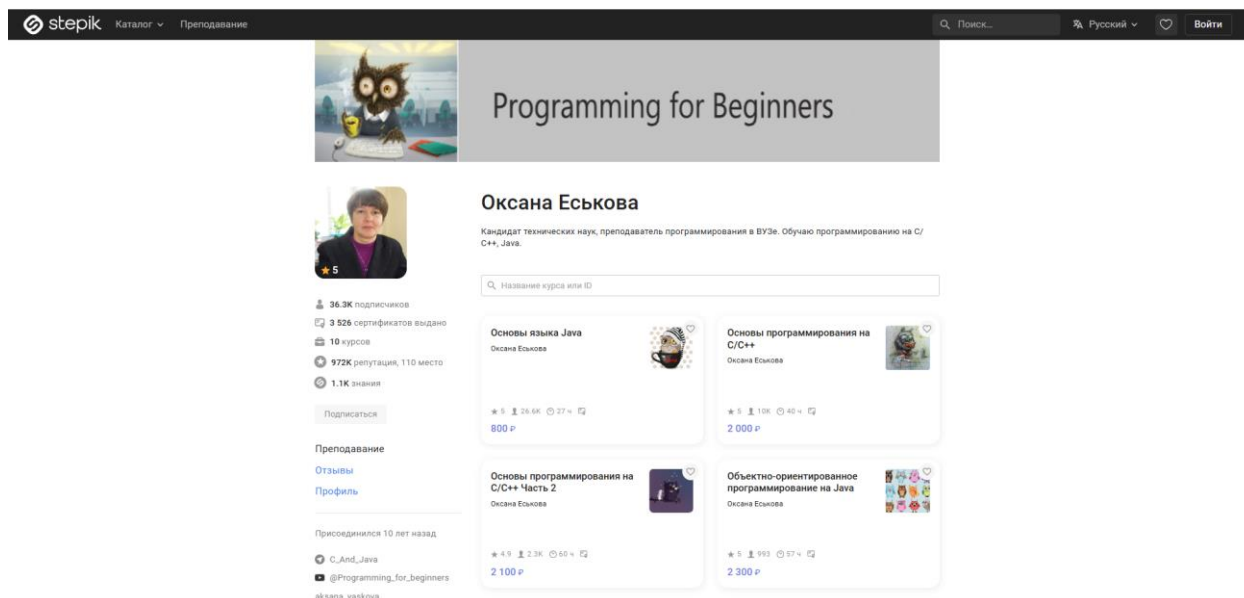


Рисунок 21 – Переход в профиль пользователя «Оксана Еськова»

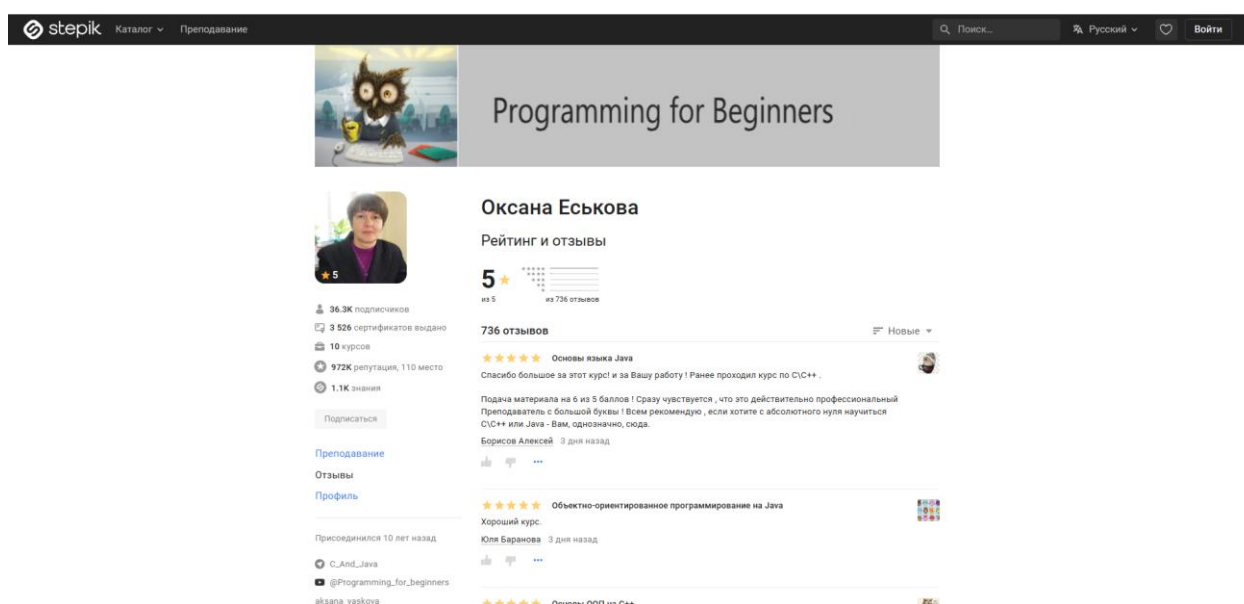


Рисунок 22 – Переход во вкладку «Отзывы»

Тест №9: Фильтрация акционных курсов

Данный тест верифицирует работу тумблера «со скидкой», проверяя, что в результатах выдачи отображаются карточки курсов, содержащие визуальный элемент зачеркнутой старой цены.

Результаты действий представлены на рисунках (см. рисунки 23, 24).

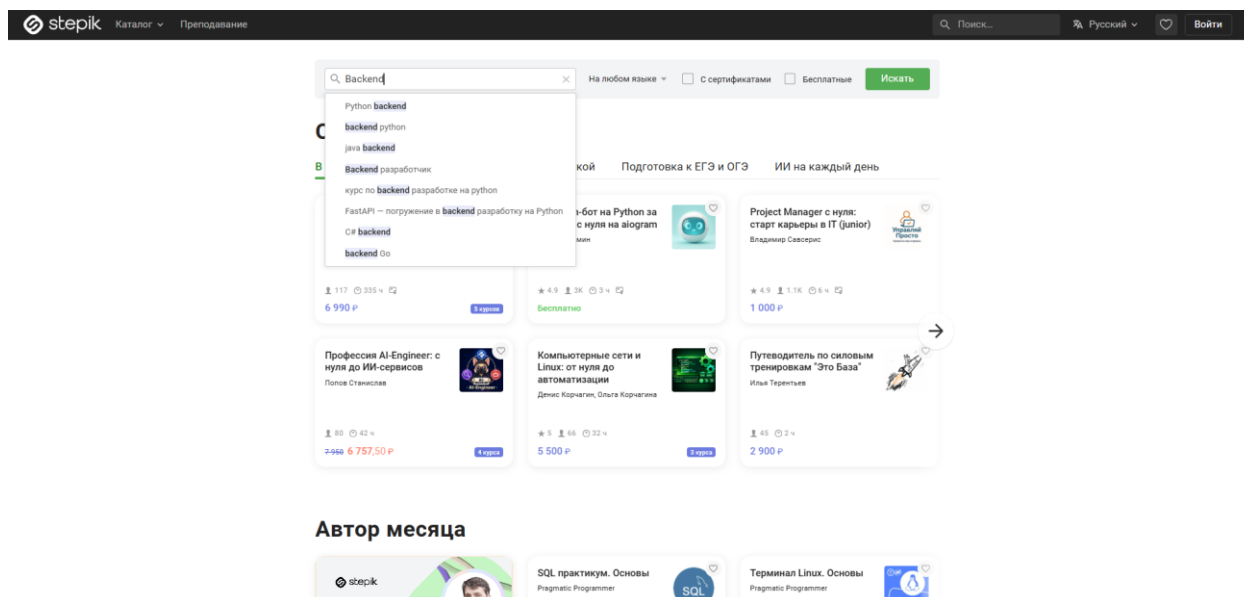


Рисунок 23 – Введение в поиск запрос «Backend»

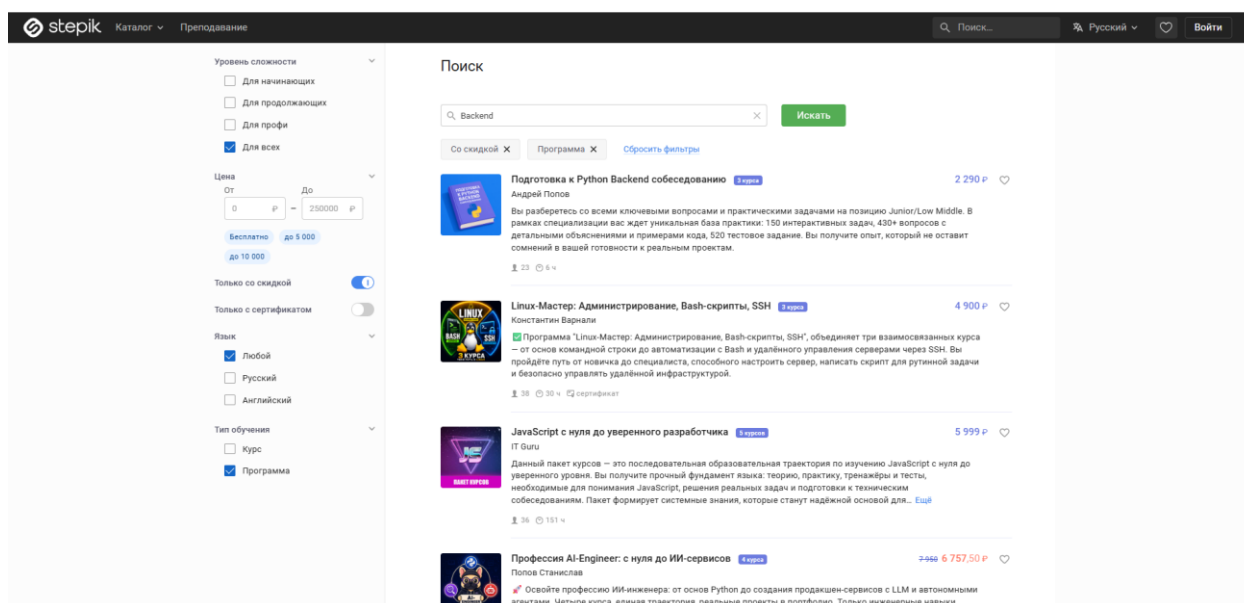


Рисунок 24 – Применение чекбоксов «Только со скидкой» и «Программа»

Тест №10: Добавление курса в избранное из поиска

Данный тест показывает интерактивное взаимодействие с компонентами карточки, проверяя возможность добавления курса в список желаемого (wishlist) и мгновенное изменение состояния иконки «сердечко».

Результаты действий представлены на рисунках (см. рисунки 25 – 27).

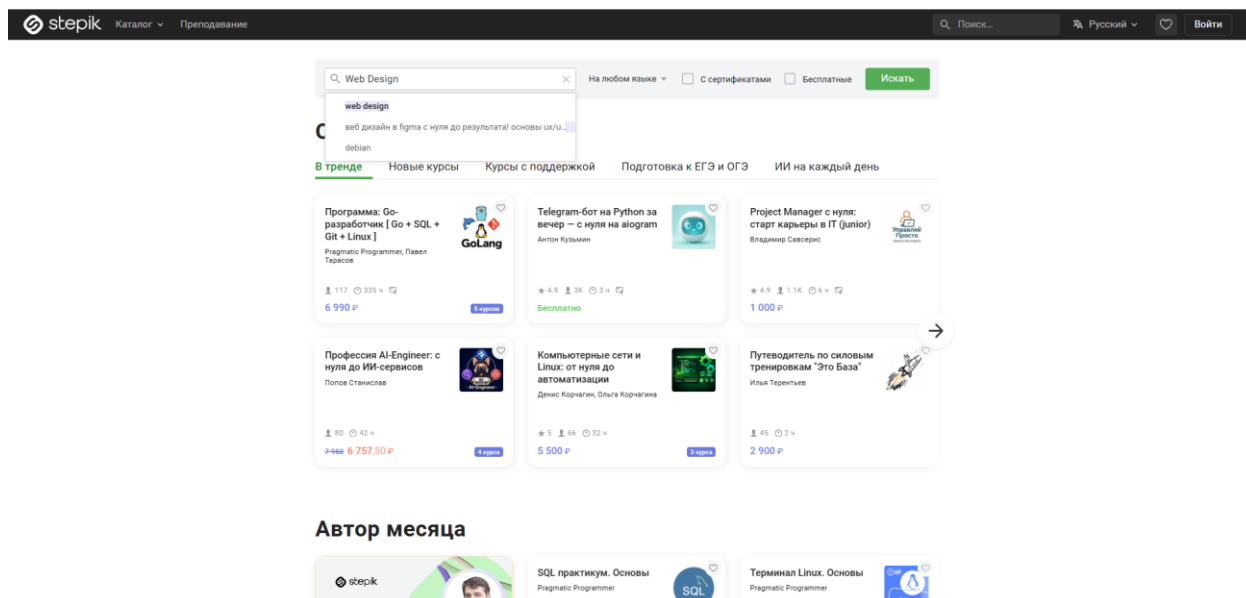


Рисунок 25 – Введение в поиск запрос «Web Design»

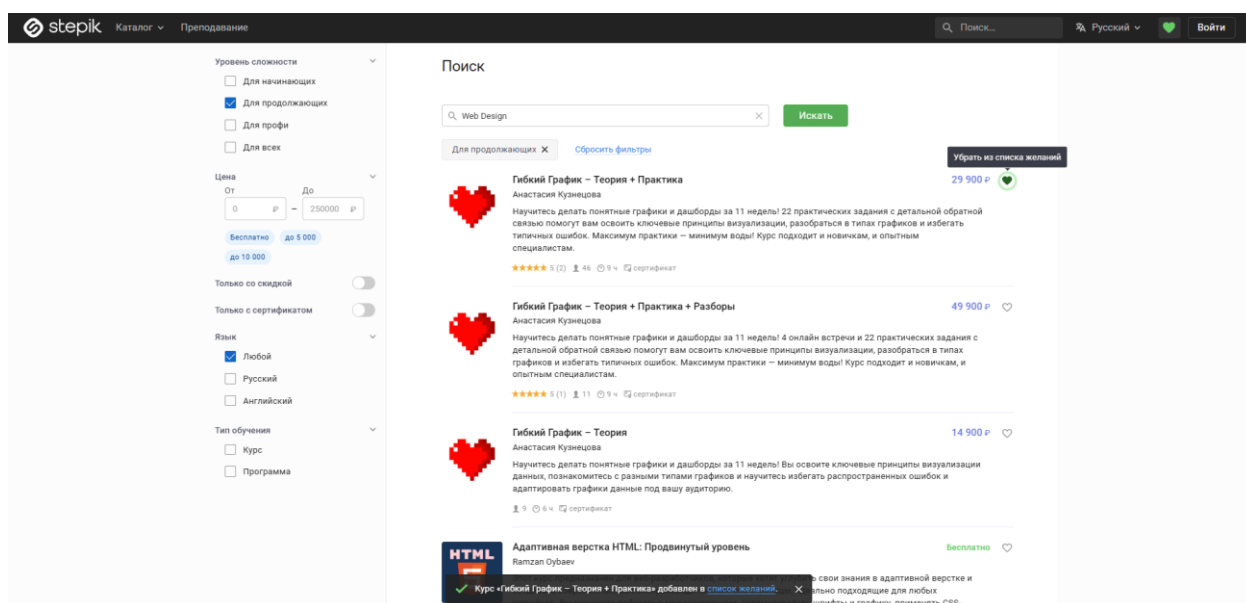


Рисунок 26 – Добавление первого курса в избранное

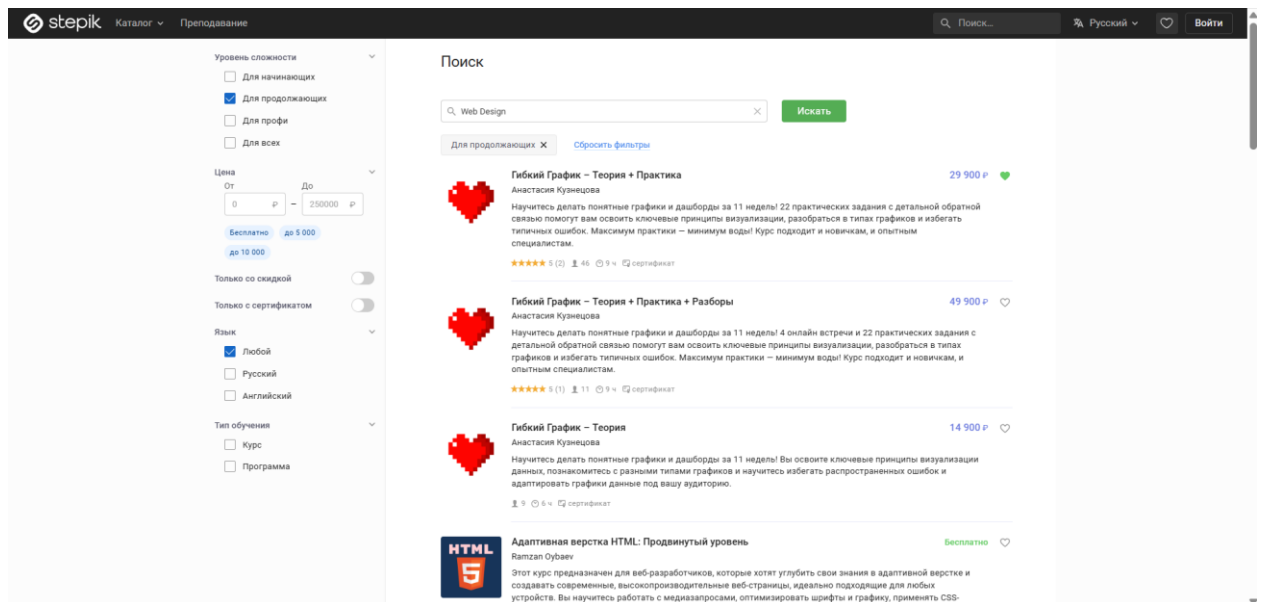


Рисунок 27 – Показ того, что первый курс в избранном (сердечко стало закрашенным)

Тесты поделены на два блока: поиск и фильтрация.

Результаты выполнения первой группы тестов представлены на рис. 28:

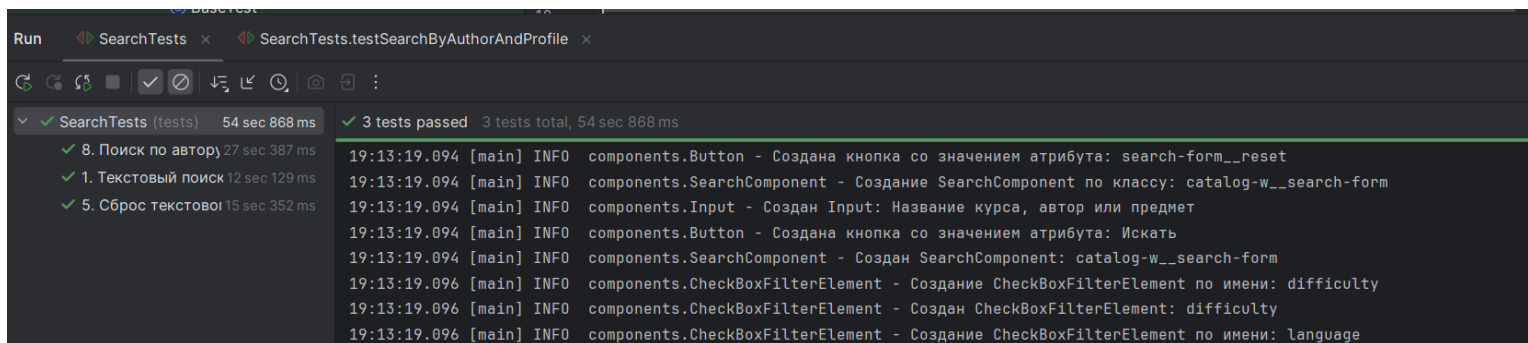


Рисунок 28 – Результаты первого блока

Результаты выполнения второй группы тестов представлены на рис. 29

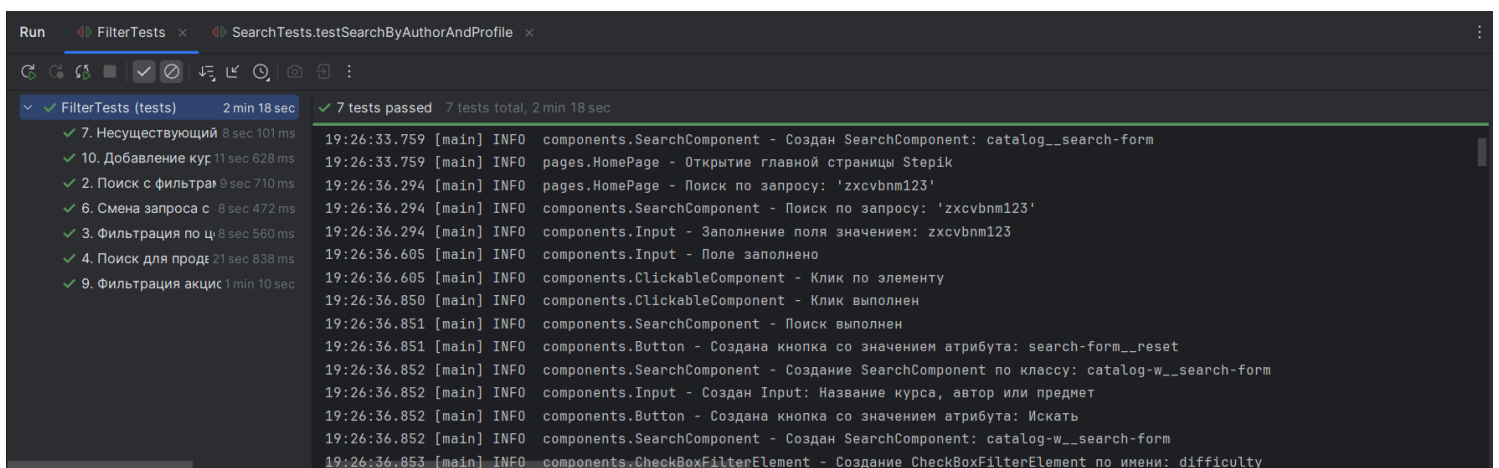


Рисунок 29 – Результаты второго блока

2 РЕЗУЛЬТАТЫ РАБОТЫ

В данном разделе приведено описание архитектуры разработанной системы автоматизированного тестирования. В основу проекта заложен паттерн проектирования Page Object Model (POM), который позволяет разделить логику тестов и описание элементов пользовательского интерфейса, обеспечивая высокую расширяемость и удобство поддержки кода.

2.1. Описание программных классов и их методов

Программный комплекс разделен на несколько логических пакетов: компоненты, страницы, тесты и вспомогательные утилиты.

Пакет **components** (UI элементы):

1. **BaseComponent:** Абстрактный базовый класс, представляющий собой обертку над SeleniumElement. Содержит общую логику поиска элементов по XPath [5] и методы доступа к ним.
2. **ClickableComponent:** Интерфейс, определяющий стандартное поведение для кликабельных элементов с обработкой ожидания видимости и логированием ошибок.
3. **Button:** Специализированный класс для работы с кнопками. Реализует интерфейс ClickableComponent. Содержит методы создания кнопки по названию (byName) и по CSS-классу (byClass).
4. **Input:** Специализированный класс для работы с полями ввода. Содержит методы создания поля по placeholder (byPlaceHolder), заполнения поля (fill) и получения значения (getValue).
5. **CheckBox:** Специализированный класс для работы с чекбоксами. Реализует интерфейс ClickableComponent. Содержит метод создания чекбокса по значению data-qa-value (byValue).
6. **CheckBoxFilterElement:** Класс для работы с группой чекбоксов внутри блока фильтрации. Содержит методы создания фильтра по имени (byName) и получения чекбокса по значению внутри группы (getCheckBoxByValue).

7. **SearchComponent:** Составной компонент, объединяющий строку поиска и кнопку отправки запроса. Содержит методы создания по CSS-классу (byClass), выполнения поиска (search) и получения текущего значения поля (getCurrentInputValue).
8. **FilterComponent:** Составной компонент, инкапсулирующий логику работы со всеми фильтрами на странице поиска: фильтрация по цене, сложности, языку и типу курса. Содержит методы применения чекбоксов (filterByCheckBox), ценового фильтра (filterByMinPrice, filterByMaxPrice), тумблеров (filterByToggler) и проверки состояния чекбокса (isCheckBoxSelected).
9. **PriceFilterElement:** Компонент для работы с ценовым фильтром. Содержит поля ввода минимальной и максимальной цены. Содержит методы получения полей ввода (getminValueInput, getMaxValueInput).
10. **ToggleFilterElement:** Класс для работы с тумблерами-переключателями. Содержит методы создания по имени (byName), переключения (toggle) и проверки активности (isActive).
11. **CourseCardItem:** Класс для работы с отдельной карточкой курса в поисковой выдаче. Реализует методы добавления в избранное (addToWishlist), клика по первому автору (clickFirstAuthor), получения заголовка (getTitle), проверки старой цены (hasOldPrice) и проверки наличия в избранном (isFavorite).
12. **CourseCardText:** Класс для работы с текстовым элементом внутри карточки курса (название или автор). Реализует интерфейс ClickableComponent. Содержит метод получения текста (getText).

Пакет pages (Page Objects):

1. **BasePage:** Абстрактная основа для всех страниц, содержащая общие параметры ожидания и методы инициализации.
2. **HomePage:** Описывает главную страницу каталога и иницирует процесс поиска.

3. `SearchResultsPage`: Центральный класс, управляющий результатами поиска, применением фильтров и навигацией по страницам (пагинацией).

4. `CoursePage` и `ProfilePage`: Классы для взаимодействия со страницей конкретного курса и профилем автора соответственно.

Пакет `helpers` и `tests`:

1. `WindowHandler`: Вспомогательный класс для управления окнами и вкладками браузера.

2. `BaseTest`: Класс конфигурации, отвечающий за настройку драйвера перед запуском тестов и закрытие сессии после их завершения.

3. `SearchTests`: Основной тестовый класс, содержащий сценарии текстового поиска, проверки пустых запросов, поиска со спецсимволами и навигации по результатам выдачи.

4. `FilterTests`: Тестовый класс, объединяющий сценарии проверки различных фильтров (цена, язык, сложность, акции). Вынесение этих тестов в отдельный класс позволило логически разделить функционал поиска и фильтрации.

5. `TestConstants`: Класс-хранилище статических констант. В него вынесены все «магические числа» (таймауты, ожидаемые тексты, тестовые наборы данных), что обеспечивает удобство редактирования тестов без изменения программной логики.

2.2. UML-диаграмма классов

На рисунке (см. рис. 30) приведена UML-диаграмма, отображающая структуру классов, их иерархию наследования, реализацию интерфейсов и взаимосвязи между компонентами и страницами.

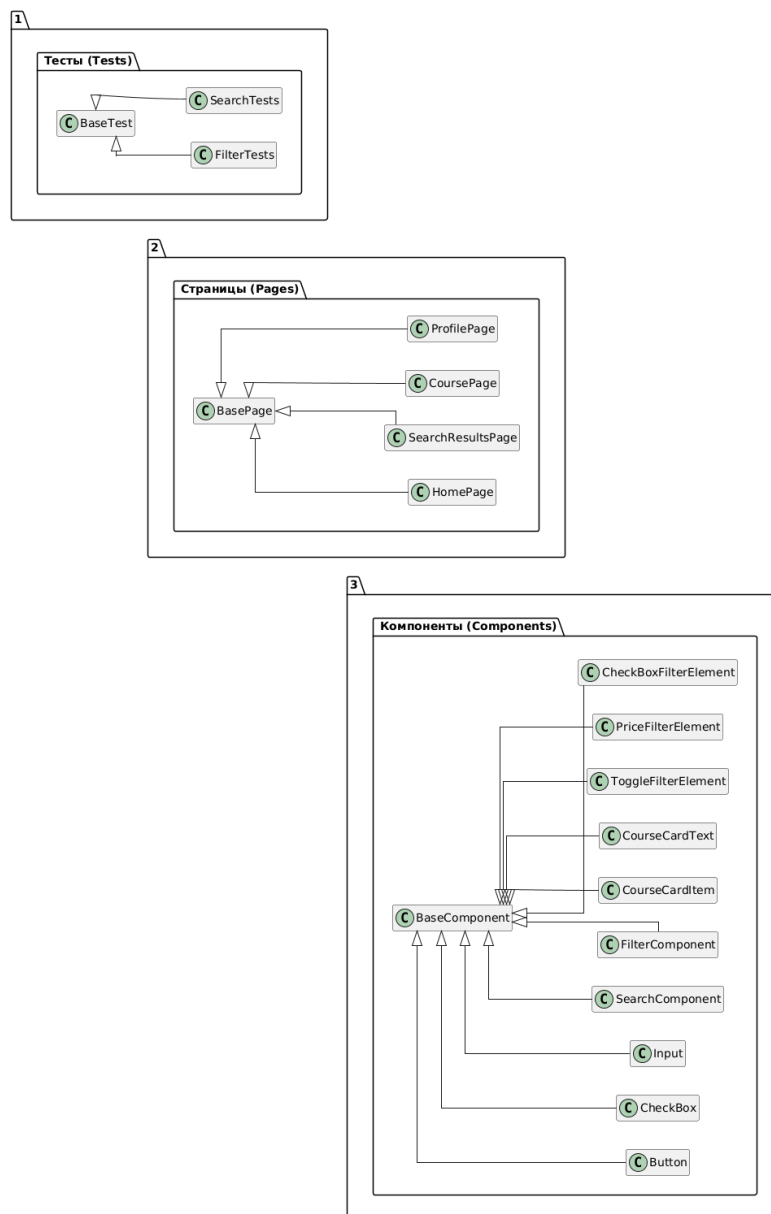


Рисунок 30 - UML-диаграмма

2.3. Реализация взаимодействия компонентов

Взаимодействие между классами построено по иерархическому принципу. Тестовые методы обращаются к методам классов страниц (Page Objects). Страницы, в свою очередь, делегируют управление специфическими блоками интерфейса составным компонентам (Components).

Например, при вызове метода `applyMinPriceFilter()` в классе `SearchResultsPage` происходит обращение к вложенному объекту `FilterComponent`, который взаимодействует с объектом `Input`. Такой

подход позволяет изменять локаторы элементов в одном месте (в классе компонента) без необходимости править код тестов.

3 ИНДИВИДУАЛЬНАЯ РАБОТА

В ходе работы над проектом автоматизации тестирования образовательного веб-ресурса Stepik мной была выполнена часть задач, связанных с настройкой инфраструктуры проекта, разработкой компонентов пользовательского интерфейса (компоненты страниц) и поддержанием единого архитектурного подхода.

Конкретные задачи, выполненные мною лично:

1. Базовая настройка проекта

Мной была выполнена первоначальная настройка проекта автоматизированного тестирования:

- создан Maven-проект;
- подключены и настроены основные зависимости, включая Selenide, JUnit 5;
- принято участие в проектировании общей архитектуры тестового фреймворка. А именно реализованы базовые классы BaseComponent, BasePage, BaseTest, которые содержат необходимый для всех своих дочерних классов функционал (для UI-компоненты это получение их Selenide элемента, для страницы это метод создания страницы без использования new, для тестов это настройка браузера, а также открытие/закрытие браузера перед/после выполнением(я) каждого теста)

2. Разработка компонентов страницы

Мной были реализованы все классы, расположенные в пакете components, предназначенные для работы с повторно используемыми элементами пользовательского интерфейса платформы.

В рамках реализации компонентов были:

- определены локаторы элементов интерфейса;
- реализованы методы взаимодействия с элементами;
- обеспечена возможность повторного использования компонентов на различных страницах проекта;

- соблюдены принципы Page Object Model и компонентно-ориентированного подхода.

3. Добавление документации к компонентам

Для всех классов пакета `components` была подготовлена JavaDoc-документация, включающая:

- описание назначения компонентов;
- пояснения к основным методам;

Наличие документации повысило читаемость кода и упростило понимание кода другими участниками.

4. Участие в разработке чек-листа

Мной было принято участие в формировании чек-листа проекта, включающего сценарии тестирования основных пользовательских функций платформы Stepik.

Работа включала:

- анализ пользовательских сценариев;
- уточнение ожидаемых результатов выполнения тестов.

5. Контроль соблюдения архитектурных принципов

В процессе разработки мной выполнялась проверка структуры проекта (проверка изменений, вносимых другими участниками, непосредственное участие в одобрении этих изменений) на соответствие выбранной архитектуре.

Основное внимание было направлено на следующие аспекты:

- корректное использование компонентов страниц;
- поддержание единообразия структуры классов.

6. Рефакторинг UI-компонентов

Мной была проведена работа по унификации структуры классов в пакете `components`. Конкретные действия по рефакторингу:

- установлен единый порядок расположения элементов - static-поля, затем instance-поля, затем конструкторы, затем static-методы, затем public-методы. Это позволило сделать код более читаемым и удобным для сопровождения;

- все Selenide элементы и числовые константы были вынесены в static поля, что позволило избежать дублирования локаторов, упростить изменение этих элементов при необходимости, а также повысило читаемость кода;
- удалены неиспользуемые импорты и методы. Это позволило сократить объём кода и повысить его чистоту.

4 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В ходе реализации учебной практики был сформирован технологический стек, обеспечивающий высокую надежность и удобство поддержки разрабатываемого решения. В качестве основного языка программирования был выбран Java, что позволило в полной мере реализовать объектно-ориентированный подход и выстроить четкую архитектуру проекта с разделением ответственности между классами страниц, компонентов и тестов. Управление зависимостями и сборка проекта осуществлялись с помощью инструмента Maven.

Для автоматизации взаимодействия с интерфейсом платформы Stepik применялся фреймворк Selenide. Выбор данного инструмента обусловлен его эффективностью при работе с динамическими элементами, а также наличием встроенных механизмов неявных ожиданий, что существенно повышает стабильность тестов. Структура тестовых сценариев и их запуск реализованы на базе библиотеки JUnit 5, а для мониторинга хода выполнения и оперативного выявления ошибок в проект интегрирована система логирования Log4j2.

Применение выбранных технологий позволило создать масштабируемый тестовый фреймворк и успешно автоматизировать проверку ключевых функций поиска и фильтрации каталога Stepik.

5 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на <ваша_оценка>.

ЗАКЛЮЧЕНИЕ

В ходе выполнения учебной практики были выполнены все поставленные задачи и достигнута основная цель, связанная с разработкой системы автоматизированного тестирования образовательной платформы Stepik. В рамках первой задачи был проведён анализ функциональных возможностей платформы, в результате которого были определены наиболее значимые пользовательские сценарии, связанные с поиском и выбором образовательных курсов. По итогам анализа был сформирован чек-лист, включающий десять тестовых сценариев, охватывающих как базовые функции поиска и навигации, так и работу с различными фильтрами, позволяющими отбирать курсы по языку, уровню сложности и другим параметрам. Разработанный набор проверок обеспечил достаточное покрытие ключевого функционала платформы.

Для решения второй задачи был использован объектно-ориентированный подход на языке Java. В основу тестового фреймворка был положен паттерн Page Object Model (POM), обеспечивающий разделение логики тестов и UI-элементов. Также, благодаря применению компонентно-ориентированного подхода были созданы универсальные классы для работы с фильтрами и строкой поиска, что значительно повысило гибкость системы и упростило ее дальнейшую поддержку. Такое решение повысило модульность проекта, упростило сопровождение кода и обеспечило возможность дальнейшего расширения функциональности тестовой системы.

Третья задача, включающая программную реализацию тестов и подготовку документации, была выполнена с использованием библиотек Selenide и JUnit 5. Применение Selenide позволило организовать стабильное взаимодействие с элементами веб-интерфейса и сократить объём служебного кода благодаря встроенным механизмам ожидания. Для контроля процесса выполнения тестов в проект была интегрирована система логирования Log4j2, обеспечивающая регистрацию выполняемых действий и возникающих событий. Кроме того, была подготовлена техническая документация проекта,

включающая описание основных программных модулей и UML-диаграммы классов, отражающие структуру и взаимосвязи компонентов разработанной системы.

В итоге, выполнение проекта способствовало закреплению практических навыков в области проектирования программных систем, применения объектно-ориентированных принципов разработки и использования современных инструментов автоматизации тестирования. Поставленная цель по созданию надёжного и масштабируемого решения для автоматизированной проверки функциональности платформы была достигнута в полном объёме. В рамках работы кодовая база была приведена к единому стилю оформления, оптимизирована путём удаления неиспользуемых элементов и дополнена необходимой документацией. По завершении проекта исходный код и сопутствующие материалы были размещены в репозитории GitHub [6], что обеспечило централизованное хранение проекта, контроль версий и возможность совместной работы с разработанным решением.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Fowler, M. PageObject [Электронный ресурс].
URL: <https://martinfowler.com/bliki/PageObject.html> (дата обращения: 07.07.2026).
2. Selenide: Concise UI Tests in Java [Электронный ресурс].
URL: <https://selenide.org/> (дата обращения: 08.07.2026).
3. JUnit 5 User Guide [Электронный ресурс].
URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 07.07.2026).
4. Apache Log4j2 Manual [Электронный ресурс]. URL: <https://logging.apache.org/log4j/2.x/manual/index.html> (дата обращения: 10.07.2024).
5. Введение в Xpath [Электронный ресурс]
URL: <https://ru.hexlet.io/blog/posts/xpath> (дата обращения: 07.07.2026).
6. <https://github.com/vyixwq/Stepik-practice>